Python Exercise — Simple Traffic Assignment

Jiwon Kim, The University of Queensland

2026

Contents

Python Exercise — Simple Traffic Assignment **2**
1. Overview . . . 2
2. The Network . . . 2
3. Data Files and Python Skeleton Files . . . 2
   `nodes.csv` — Node IDs and coordinates . . . 3
   `net.csv` — Link data and BPR function parameters . . . 3
   `trips.csv` — OD demand . . . 4
4. Load the Data - Task 1 . . . 4
5. Initialisation . . . 5
6. Find Shortest Paths for All OD-Pairs — Task 2 . . . 7
   Step 1 — Define `shortestPath()` in `w5_task_functions.py` . . . 7
   Step 2 — Import and call the function in the main script . . . 8
7. Assign Demand to Shortest Paths and Update Link Flows — Task 3 . . . 9
8. Update Link Costs using BPR — Task 4 . . . 10

Python Exercise — Simple Traffic Assignment

1. Overview

In this exercise we perform a simple **static traffic assignment** model: using Dijkstra's algorithm to find the shortest path for each origin-destination (OD) pair, then allocating OD demand to those shortest paths.

We use a four-node, five-link toy network, as shown in the figure below.

You will load link, node, and OD demand data from CSV files, compute shortest paths for all OD pairs, allocate demand to those shortest paths, and update link flows. Then, you will update link costs (travel times) using the **BPR link performance function** so you can see how congestion affects link travel time after assignment.

---

2. The Network

The four-node, five-link network used in this exercise:

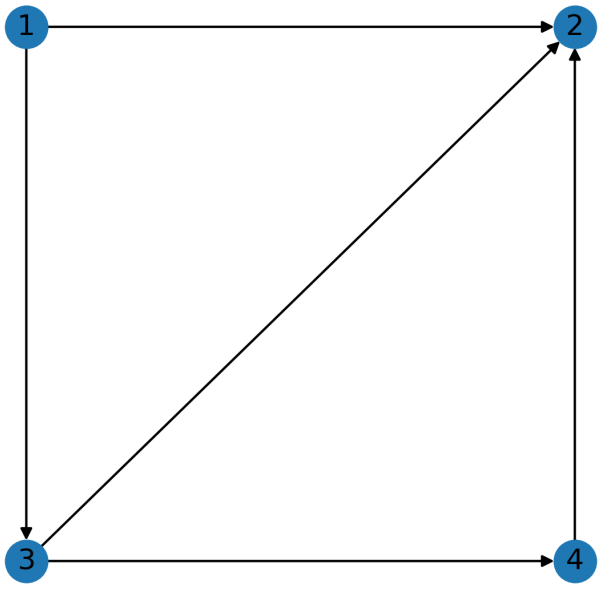

Figure 1: Road network

---

3. Data Files and Python Skeleton Files

Download the following files from Blackboard (Content > Week 5) and place them in your project directory: - `toy_net.zip` — the network data. - `w5_task_main.py` — the main script you will edit to complete the tasks. - `w5_task_functions.py` — a helper module containing function skeletons you will fill in.

Then set up the data files as follows: 1. In your project directory, create a `data/` folder if it does not already exist. 2. Move `toy_net.zip` into `data/`. 3. Unzip `toy_net.zip` inside `data/` so that the extracted folder is `data/toy_net/`. 4. Check that `nodes.csv`, `net.csv`, and `trips.csv` are directly inside `data/toy_net/`. 5. After confirming the files are in place, you may delete `toy_net.zip`.

Your project structure should look like this:

```
project_directory/
|-- w5_task_main.py
|-- w5_task_functions.py
|-- data/
|   |-- toy_net/
|       |-- nodes.csv
|       |-- net.csv
|       |-- trips.csv
```

`nodes.csv` — Node IDs and coordinates

Each row represents one node. The `x` and `y` columns are the node's position in the network diagram — they are used for plotting only and do not affect the algorithm.

node	x	y
1	0	1
2	1	1
3	0	0
4	1	0

`net.csv` — Link data and BPR function parameters

Each row represents a directed link. In addition to geometry (`length`), each link has the parameters of the **Bureau of Public Roads (BPR) link performance function**, which models how link travel time increases with link traffic flow.

init_node	term_node	capacity	length	free_flow_time	b	power
1	2	5.0	2.0	4.0	1	1
1	3	15.0	1.0	2.0	1	1
3	2	10.0	1.5	3.0	1	1
3	4	20.0	1.0	2.0	1	1
4	2	15.0	1.0	2.0	1	1

The BPR function for link (i, j) is defined as:

$$t_{ij}(x_{ij}) = \text{free\_flow\_time}\left[1 + b\left(\frac{x_{ij}}{\text{capacity}}\right)^{\text{power}}\right]$$

where:

- t_{ij} is the travel time on link (i, j) when the flow is x_{ij}
- x_{ij} is the current link flow (*vehicles per unit time*)
- `free_flow_time` is the travel time when the link is empty
- `capacity` is the link's maximum flow (*vehicles per unit time*)
- `b` and `power` are calibration parameters

In this toy network, every link has `b = 1` and `power = 1`, so the BPR function simplifies to:

$$t_{ij}(x_{ij}) = \text{free\_flow\_time}\left(1 + \frac{x_{ij}}{\text{capacity}}\right)$$

`trips.csv` — OD demand

Each row gives the travel demand for one origin–destination (OD) pair, where demand is in *vehicles per unit time*. In this toy network, there are 6 OD pairs with non-zero demand, each with demand of 5.0 vehicles per unit time. The file lists only the non-zero demand pairs; any OD pair not listed has zero demand.

orig	dest	demand
1	2	5.0
1	3	5.0
1	4	5.0
3	2	5.0
3	4	5.0
4	2	5.0

---

4. Load the Data - Task 1

Load each CSV file into a Pandas DataFrame using `pd.read_csv(..., sep=',')`. Then set the appropriate index on each DataFrame using `set_index(..., inplace=True, drop=False)`.

- `df_nodes` — load `nodes.csv`; set `'node'` as the index.
- `df_links` — load `net.csv`; set `['init_node', 'term_node']` as a MultiIndex.
- `df_ods` — load `trips.csv`; set `['orig', 'dest']` as a MultiIndex.

Print all three DataFrames when done.

```
# %% Imports and File Paths
import pandas as pd
import numpy as np

# Tells NumPy to use the legacy print format, which prints plain numbers instead of
# verbose types (e.g. '3.0' instead of 'np.float64(3.0)')
np.set_printoptions(legacy="1.25")

# File paths to the toy_net data
# (update these if your data files are in a different location).
node_file = "data/toy_net/nodes.csv"
link_file = "data/toy_net/net.csv"
demand_file = "data/toy_net/trips.csv"

# %% Load the Data - Task 1

# TODO: load nodes.csv into df_nodes and set 'node' as the index.
df_nodes = None # Replace 'None' with your code.
```

```python
# TODO: load net.csv into df_links and set ['init_node', 'term_node'] as a MultiIndex
df_links = None # Replace 'None' with your code.

# TODO: load trips.csv into df_ods and set ['orig', 'dest'] as a MultiIndex.
df_ods = None # Replace 'None' with your code.

print("--- Network Data ---")
print("df_nodes:")
print(df_nodes)
print("\ndf_links:")
print(df_links)
print("\ndf_ods:")
print(df_ods)
```

Expected output:

```
--- Network Data ---
df_nodes:
      node  x  y
node
1        1  0  1
2        2  1  1
3        3  0  0
4        4  1  0

df_links:
                     init_node  term_node  capacity  length  free_flow_time  b  power
init_node term_node
1         2                  1          2       5.0     2.0             4.0  1      1
          3                  1          3      15.0     1.0             2.0  1      1
3         2                  3          2      10.0     1.5             3.0  1      1
          4                  3          4      20.0     1.0             2.0  1      1
4         2                  4          2      15.0     1.0             2.0  1      1

df_ods:
            orig  dest  demand
orig dest
1    2         1     2     5.0
     3         1     3     5.0
     4         1     4     5.0
3    2         3     2     5.0
     4         3     4     5.0
4    2         4     2     5.0
```

---

5. Initialisation

Set up the **forward star** dictionary and add **initial columns** to `df_links` and `df_ods` that the traffic assignment loop will read and update:

- `df_links["cost"]` — current link travel time, initialised to `free_flow_time`. Updated by the

BPR function after each iteration.

- `df_links["flow"]` — accumulated vehicle flow on each link, initialised to `0.0`. Incremented as demand is assigned to paths.
- `df_ods["shortest_path_time"]` — travel time of the shortest path for each OD pair, initialised to `0.0`.
- `df_ods["shortest_path"]` — the shortest path as a list of node IDs, initialised to `None` (rather than `0.0`) so the column can hold list objects.

```
# %% Initialisation

# Construct the forward star dictionary to store the downstream nodes for each node.
forward_star = {}

# Loop through all nodes in df_nodes to initialise forward_star with empty lists.
for row in df_nodes.itertuples(index=False):
    forward_star[row.node] = []

# Loop through all links in df_links to populate the forward_star dictionary.
for row in df_links.itertuples(index=False):
    forward_star[row.init_node].append(row.term_node)

print("\nforward_star:", forward_star)

# Create the 'cost' column to store link travel time
# Initialise the values to 'free_flow_time'
df_links["cost"] = df_links["free_flow_time"]

# Create the 'flow' column to store link flow values
# Initialise the values to 0.0
df_links["flow"] = 0.0

# Create the 'shortest_path_time' column in df_ods to store
# the shortest path travel time for each OD pair
df_ods["shortest_path_time"] = 0.0

# Create the 'shortest_path' column in df_ods to store the list of node IDs
# for the shortest path for each OD pair;
# Initialise with 'None' to store 'object' data type, such as lists or strings
df_ods["shortest_path"] = None

print("\n--- Prepared for Assignment ---")
print("\nforward_star:", forward_star)
print("\ndf_links:")
print(df_links)
print("\ndf_ods:")
print(df_ods)
```

Output:

```
--- Prepared for Assignment ---

forward_star: {1: [2, 3], 2: [], 3: [2, 4], 4: [2]}

df_links:
                init_node  term_node  capacity  length  free_flow_time  b  power  cost  flow
```

```
init_node term_node
1         2             1          2       5.0     2.0             4.0  1      1   4.0   0.0
          3             1          3      15.0     1.0             2.0  1      1   2.0   0.0
3         2             3          2      10.0     1.5             3.0  1      1   3.0   0.0
          4             3          4      20.0     1.0             2.0  1      1   2.0   0.0
4         2             4          2      15.0     1.0             2.0  1      1   2.0   0.0
```

```
df_ods:
           orig  dest  demand  shortest_path_time shortest_path
orig dest
1    2        1     2     5.0                 0.0          None
     3        1     3     5.0                 0.0          None
     4        1     4     5.0                 0.0          None
3    2        3     2     5.0                 0.0          None
     4        3     4     5.0                 0.0          None
4    2        4     2     5.0                 0.0          None
```

---

6. Find Shortest Paths for All OD-Pairs — Task 2

Step 1 — Define `shortestPath()` in `w5_task_functions.py`

We want to find the shortest path from every origin node to every other node in the network. In the single-origin case, Dijkstra's shortest path algorithm can be implemented directly in the main script. Here, we extend that approach to **all** origin nodes.

To keep the main script clean and reusable, we move the shortest path algorithm into a **function** named `shortestPath()` defined in a separate file called `w5_task_functions.py`. This lets us call it with one line for each origin instead of repeating the full algorithm each time.

A skeleton of the `shortestPath()` function is already provided in `w5_task_functions.py` — your task is to fill in the body of the function using your Dijkstra implementation.

You can remove all `print()` calls from inside the function to avoid cluttering the output when calling it multiple times.

The `shortestPath()` function takes four inputs:

Parameter	Type	Description
`orig_node`	`int`	Origin node ID
`df_nodes`	DataFrame	Node table
`df_links`	DataFrame	Link table — uses `df_links['cost']` as travel time
`forward_star`	`dict`	Maps each node to its list of downstream nodes

And returns three outputs:

Return value	Type	Description
`L`	`dict`	Shortest path cost from `orig_node` to each node
`q`	`dict`	Backnode of each node on the shortest path
`odpath`	`dict`	Key: `(orig_node, dest_node)`, value: list of node IDs on the path

Step 2 — Import and call the function in the main script

Once `shortestPath()` is defined, import `w5_task_functions.py` in your main script as a module using alias `fn` (i.e. `import w5_task_functions as fn`). Then, functions defined in that file can be called using the `fn.` prefix, e.g. `fn.shortestPath(...)`.

We want to find the shortest path for each OD pair in `df_ods`. To do this, you will need two loops: one outer loop over origin nodes, and one inner loop over destination nodes for each origin:

- For the **outer loop** (origins): iterate over `df_ods["orig"].unique()` — this creates a list of unique origin nodes that actually appear in the OD table.
- For the **inner loop** (destinations): iterate over `df_ods[df_ods["orig"] == orig_node]["dest"]` — this filters `df_ods` to rows where `orig` matches the current origin, then retrieves the `dest` column, giving only the destinations paired with that origin.

Note that both the outer and inner loops iterate only over the nodes that **actually appear in `df_ods`**, not all nodes in `df_nodes`. This is more efficient than iterating over all nodes, and also ensures we only compute paths for OD pairs that exist in the demand table.

In the outer loop over origin nodes, call the shortest path function as `fn.shortestPath(...)` for each origin. It will compute the shortest path from the origin to all other nodes and return the results in `L`, `q`, and `odpath`. Then, in the inner loop over the destination nodes for that origin, update the `shortest_path_time` and `shortest_path` columns in `df_ods` for each OD pair.

```python
# %% Find Shortest Paths for All OD-Pairs - Task 2
import w5_task_functions as fn

# TODO: loop over unique origin nodes using df_ods["orig"].unique().
# For each orig_node, call fn.shortestPath(...) to get L, q, and odpath.
# Then loop over destination nodes for this origin using
# df_ods[df_ods["orig"] == orig_node]["dest"].
# For each dest_node, if the given OD-pair key is in odpath,
# update df_ods's 'shortest_path_time' and 'shortest_path' columns for this OD pair.

print("\n--- OD Shortest Paths ---")
print(df_ods)
```

Expected output:

```
--- OD Shortest Paths ---
           orig  dest  demand  shortest_path_time shortest_path
orig dest
1    2        1     2     5.0                 4.0        [1, 2]
     3        1     3     5.0                 2.0        [1, 3]
     4        1     4     5.0                 4.0     [1, 3, 4]
3    2        3     2     5.0                 3.0        [3, 2]
     4        3     4     5.0                 2.0        [3, 4]
4    2        4     2     5.0                 2.0        [4, 2]
```

Visual illustration of shortest-path assignment results:

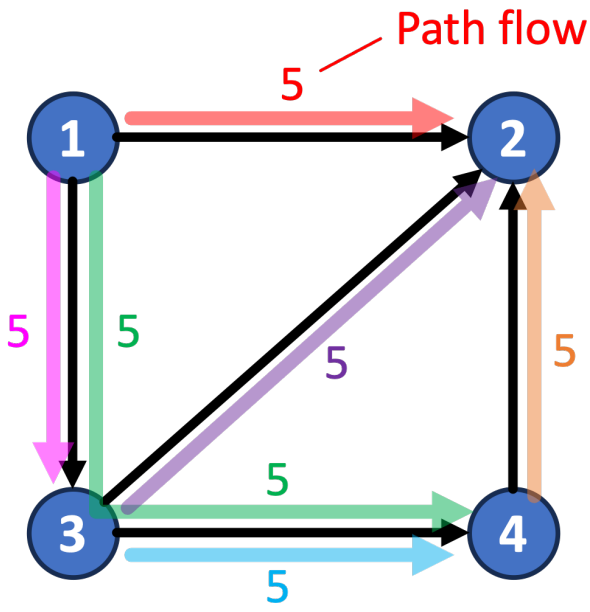

Figure 2: Shortest-path assignment results

---

7. Assign Demand to Shortest Paths and Update Link Flows — Task 3

We will implment a simple traffic assignment algorithm that assigns all demand to the shortest paths, without any iteration or feedback between flows and costs.

With the shortest paths stored in `df_ods`, assign each OD pair's full travel demand to the links along the shorteat path for that OD pair. The idea is: every traveller between an origin and a destination takes the shortest path, so the entire demand for that OD pair is loaded onto all links that make up that path.

Because multiple OD pairs may share links, **link flows must accumulate shortestpath flows (demand) across all OD pairs** — you are adding demand to `df_links['flow']`, not overwriting it.

The figure above shows how each OD demand would be loaded onto its shortest path and how the resulting path flows accumulate on shared links.

Your task: Loop over `df_ods` using `itertuples()`. For each OD row:

1. Retrieve `demand` and `shortest_path` from the row.
2. Loop over consecutive node pairs along the path — each pair identifies one link, i.e., for `shortest_path = [1, 3, 4]`, the node pairs are `(1, 3)` and `(3, 4)`.
3. For each node pair (i.e., link), add the demand to that link's flow, df_links's `flow` column.

```
# %% Assign Demand to Shortest Paths and Update Link Flows - Task 3

# TODO: loop over df_ods using itertuples().
# For each row, retrieve the demand and the shortest_path.
# Then loop over consecutive node pairs to find the links along the path.
# For each node pair (i.e., link), add the demand to df_links's `flow` column.
```

```
print("\n--- Link Flows after Assignment ---")
print(df_links[["flow"]])
```

Expected output:

```
--- Link Flows after Assignment ---
                    flow
init_node term_node
1         2          5.0
          3         10.0
3         2          5.0
          4         10.0
4         2          5.0
```

---

8. Update Link Costs using BPR — Task 4

Now that `df_links['flow']` has been populated, the travel time on each link must be updated to reflect congestion. We use the **BPR function** introduced in Section 3:

$$t_{ij} = \text{free\_flow\_time}\left[1 + b\left(\frac{\text{flow}}{\text{capacity}}\right)^{\text{power}}\right]$$

Your task: loop over `df_links` and update the cost function. For each link, the BPR function parameters are available in other columns of `df_links`.

```
# %% Update Link Costs using BPR - Task 4

# TODO: loop over df_links and update df_links['cost'] using the BPR function.
# For each link, use free_flow_time, b, flow, capacity, and power from df_links.

print("\n--- Link Flows and Costs after BPR Update ---")
print(df_links[["capacity", "flow", "free_flow_time", "cost"]])
```

Expected output:

```
--- Link Flows and Costs after BPR Update ---
                    capacity  flow  free_flow_time      cost
init_node term_node
1         2              5.0   5.0             4.0  8.000000
          3             15.0  10.0             2.0  3.333333
3         2             10.0   5.0             3.0  4.500000
          4             20.0  10.0             2.0  3.000000
4         2             15.0   5.0             2.0  2.666667
```